Diseño y Análisis de Algoritmos

Ayudantía 7: Grafos: BFS, DFS y Dijkstra

Ayud. Ferran Rojas Andreu

Enunciado

Considere el grafo dirigido $G = (V, E)$ con

$$V = \{a, b, c, d, e\},$$

cuyas listas de adyacencia, *ordenadas alfabéticamente*, son:

$$\textsc{Adj}(a) = \{b, d\}, \qquad \textsc{Adj}(b) = \{c, d\}, \qquad \textsc{Adj}(c) = \{a\},$$
$$\textsc{Adj}(d) = \varnothing, \qquad \textsc{Adj}(e) = \{d\}.$$

Ejecute el algoritmo de búsqueda en profundidad (DFS) sobre G, considerando que:

- (i) el bucle externo del procedimiento DFS recorre los vértices en orden alfabético;
- (ii) cada lista de adyacencia está ordenada alfabéticamente;
- (iii) el contador global de tiempo `time` comienza en 0: al *descubrir* un vértice u se hace `time++` y se asigna $d[u] =$ `time`; al *finalizar* u se hace nuevamente `time++` y se asigna $f[u] =$ `time`.

Determine:

(a) los tiempos de descubrimiento $d[v]$ y de finalización $f[v]$ de cada vértice;

(b) la clasificación de cada arista de G en *tree edge*, *back edge*, *forward edge* o *cross edge*, justificando el tipo de cada una.

Supuesto: el grafo es conexo a partir de a salvo el vértice e, que se alcanza desde el bucle externo.

Enunciado 2: BFS, DFS y Dijkstra (CLRS)

Considere el grafo dirigido y ponderado $G = (V, E)$ con

$$V = \{s,\ a,\ b,\ c,\ d,\ z\},$$

cuyas listas de adyacencia, *ordenadas alfabéticamente*, son:

$$\begin{aligned} \textsc{Adj}(s) &= \{(a,7),\ (b,2)\}, & \textsc{Adj}(a) &= \{(c,4)\}, \\ \textsc{Adj}(b) &= \{(a,3),\ (c,5)\}, & \textsc{Adj}(c) &= \{(d,2)\}, \\ \textsc{Adj}(d) &= \{(z,2)\}, & \textsc{Adj}(z) &= \varnothing, \end{aligned}$$

donde cada par (v, w) indica el vecino v y el peso w de la arista.

Determine:

(a) ejecute BFS con fuente s, mostrando para cada vértice su distancia $d[v]$, su predecesor $\pi[v]$ y dibuje (describa) el árbol BFS;

(b) ejecute DFS sobre G (el bucle externo y las listas de adyacencia en orden alfabético), mostrando los tiempos $d[v]$, $f[v]$ y la clasificación de cada arista;

(c) ejecute Dijkstra con fuente s, mostrando el conjunto S y los valores de $d[v]$ tras cada iteración, los predecesores finales, y el camino más corto desde s hasta z con su costo.

Notas: (i) BFS ignora los pesos de las aristas; (ii) el orden de los vértices es a, b, c, d, s, z (alfabético).

Los siguientes son los algoritmos del libro de *CLRS*:

BFS

```
BFS(G, s)
1  for cada vértice u ∈ G.V \ {s}
2      u.color ← WHITE;   u.d ← ∞;   u.π ← NIL
3  s.color ← GRAY;   s.d ← 0;   s.π ← NIL
4  Q ← ∅;   ENQUEUE(Q, s)
5  while Q ≠ ∅
6      u ← DEQUEUE(Q)
7      for cada v ∈ G.Adj[u]
8          if v.color = WHITE
9              v.color ← GRAY;   v.d ← u.d + 1;   v.π ← u
10             ENQUEUE(Q, v)
11     u.color ← BLACK
```

DFS

```
DFS(G)
1  for cada vértice u ∈ G.V
2      u.color ← WHITE;   u.π ← NIL
3  time ← 0
4  for cada vértice u ∈ G.V
```

```
5       if u.color = WHITE
6           DFS-Visit(G, u)
```

DFS-Visit(G, u)

```
1   time ← time + 1;   u.d ← time;   u.color ← GRAY
2   for cada v ∈ G.Adj[u]
3       if v.color = WHITE
4           v.π ← u;   DFS-Visit(G, v)
5   u.color ← BLACK;   time ← time + 1;   u.f ← time
```

Dijkstra

Init-Single-Source(G, s) (para cada v: $d[v] \leftarrow \infty$, $d[s] \leftarrow 0$, $\pi[v] \leftarrow$ NIL)

Relax(u, v, w): si $d[v] > d[u] + w(u, v)$, entonces $d[v] \leftarrow d[u] + w(u, v)$ y $\pi[v] \leftarrow u$.

Dijkstra(G, w, s)

```
1   Init-Single-Source(G, s)
2   S ← ∅
3   Q ← G.V                                   ▷ cola de prioridad por d
4   while Q ≠ ∅
5       u ← Extract-Min(Q)
6       S ← S ∪ {u}
7       for cada v ∈ G.Adj[u]
8           Relax(u, v, w)
```